

CSCI 485 — Assignment #3

Image Compression via Block-wise SVD

Piyush Jadhav

1 Implementation Summary

This assignment implements block-wise Singular Value Decomposition (SVD) for grayscale image compression. Given a matrix A , SVD decomposes it as $A = U\Sigma V^T$. A rank- k approximation retains only the top- k singular values:

$$A_k = U_k \Sigma_k V_k^T \quad (1)$$

By the Eckart–Young theorem, A_k is the optimal rank- k approximation in the Frobenius norm — making SVD well-suited for image compression.

The pipeline works as follows:

- A 512×512 grayscale image is partitioned into non-overlapping 8×8 blocks.
- Each block is independently compressed using a rank- k SVD approximation.
- The compressed blocks are reassembled into the full reconstructed image.
- This is repeated for $k \in \{1, 2, \dots, 8\}$ to study the quality-compression trade-off.

The compression ratio and reconstruction quality are evaluated using:

$$\text{Compression Ratio} = \frac{n^2}{k(2n+1)} \quad \text{PSNR} = 10 \cdot \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \quad (2)$$

For 8×8 blocks this simplifies to $64/(17k)$.

2 Important Code Snippets

Block compression using SVD:

```
1 def compress_block(block: np.ndarray, k: int) -> np.ndarray:  
2     U, S, Vt = np.linalg.svd(block, full_matrices=False)  
3     U_k = U[:, :k]          # (8, k)  
4     S_k = S[:k]             # (k, )  
5     Vt_k = Vt[:k, :]       # (k, 8)  
6     return U_k @ np.diag(S_k) @ Vt_k
```

Listing 1: Rank-k SVD approximation of a single 8×8 block.

Full image block-wise compression:

```
1 def compress_image(img, k, block_size=8):
2     H, W = img.shape
3     result = np.zeros_like(img)
4     for i in range(0, H, block_size):
5         for j in range(0, W, block_size):
6             block = img[i:i+block_size, j:j+block_size]
7             result[i:i+block_size, j:j+block_size] =
                compress_block(block, k)
8     return np.clip(result, 0, 255)
```

Listing 2: Partition image into blocks, compress each, and recombine.

3 Analysis of Results

3.1 Original Image



Figure 1: Original grayscale input image (512×512).

3.2 Compression Ratio and Reconstruction Error

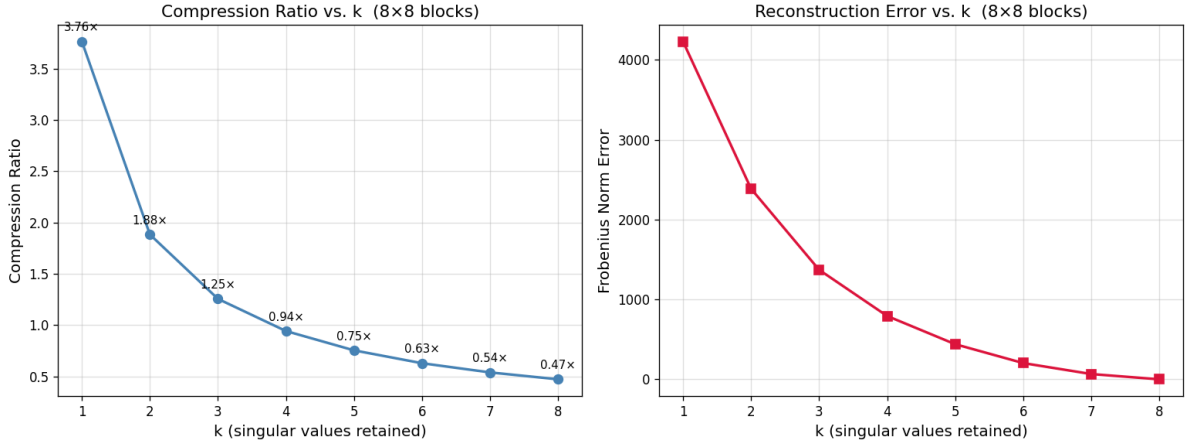


Figure 2: Compression ratio (left) and Frobenius norm error (right) vs. k for 8×8 blocks.

The compression ratio decreases hyperbolically with k following $64/(17k)$. At $k = 1$, each block is stored as only 17 values instead of 64 — a 73% reduction. The Frobenius error drops sharply from $k = 1$ to $k = 4$ and then flattens, forming a classic *elbow curve*. This reflects the rapid decay of singular values: the first few components capture most of the image energy (low-frequency structure), while later ones encode fine detail.

3.3 PSNR vs. k

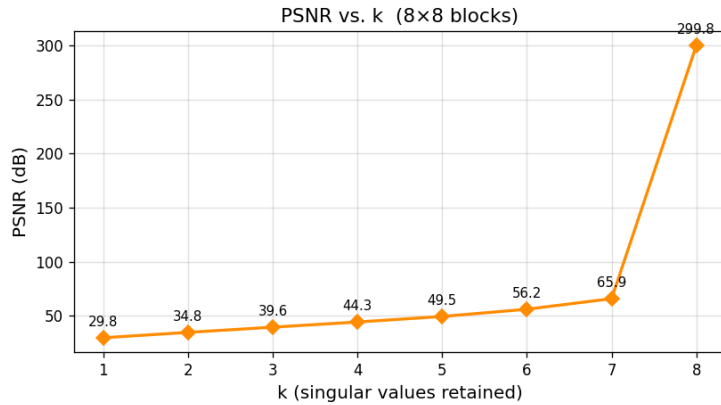


Figure 3: PSNR (dB) as a function of k for 8×8 blocks.

PSNR rises steadily from 29.8 dB at $k = 1$ to 65.9 dB at $k = 7$, then spikes to ≈ 300 dB at $k = 8$ — the full-rank case where reconstruction is numerically identical to the original. A PSNR above 30 dB is generally considered visually acceptable; even $k = 2$ achieves 34.8 dB while still compressing at 1.88 $\times$, making it the practical sweet spot.

Table 1: Compression metrics for 8×8 block-wise SVD.

k	Compression Ratio	Frobenius Error	PSNR (dB)
1	$3.76\times$	4228.88	29.8
2	$1.88\times$	2385.73	34.8
3	$1.25\times$	1370.81	39.6
4	$0.94\times$	792.71	44.3
5	$0.75\times$	439.62	49.5
6	$0.63\times$	203.24	56.2
7	$0.54\times$	66.08	65.9
8	$0.47\times$	≈ 0	299.8

Fill in Frobenius error values from notebook output.

3.4 Visual Comparison



Figure 4: Visual comparison of original and SVD-compressed images at $k = 1, 2, 4, 8$.

- **$k=1$:** The image is recognizable but blurry — fine textures (shirt pattern, brick floor) are lost. Subtle block boundaries are visible.
- **$k=2$:** A clear improvement. Edges are restored and block artefacts reduce. Still compresses at $1.88\times$.
- **$k=4$:** Visually near-identical to the original, but the compression ratio ($0.94\times$) means no storage savings.
- **$k=8$:** Perceptually identical to the original. Full-rank — no compression.

3.5 Block Size Comparison (4×4 , 8×8 , 16×16)

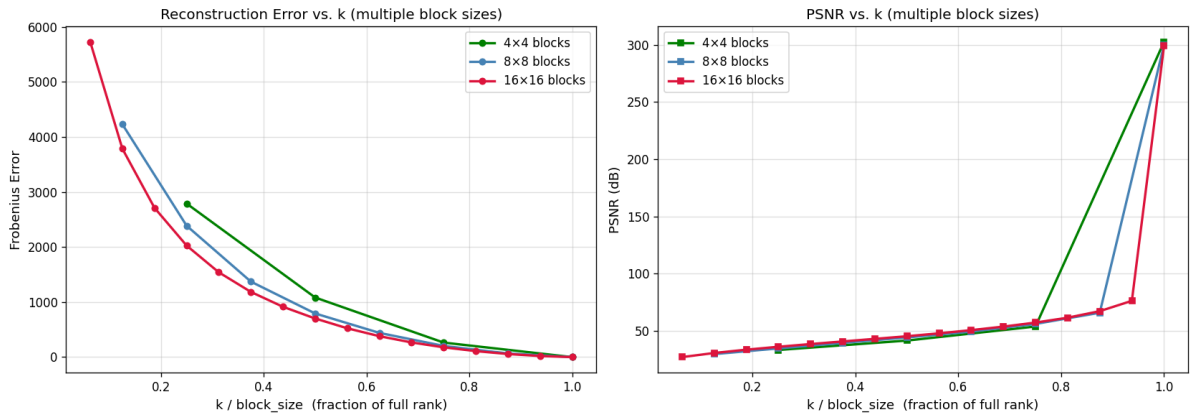


Figure 5: Frobenius error (left) and PSNR (right) vs. normalized rank $k/\text{block_size}$ across block sizes.

Table 2: Maximum compression ratio at $k = 1$ by block size.

Block Size	Original Values	Stored at $k=1$	Max Ratio
4×4	16	9	$1.78\times$
8×8	64	17	$3.76\times$
16×16	256	33	$7.76\times$

Larger blocks offer higher compression ceilings but introduce more visible block artefacts at low k , since each block captures a wider and more complex region. Smaller blocks have less spatial redundancy to exploit. The 8×8 block size strikes the best practical balance — consistent with its use in the JPEG standard.

GitHub Repository

https://github.com/nv-piyush/CSCI485_Piyush_Jadhav